HEAT Accessibility Extensions: Keyboard Navigation, Text-to-Speech, and Colour Accessibility for a Haskell IDE

Yarushah (ys321)
School of Computing
University of Kent
Canterbury, United Kingdom

Goutham (gs100)
School of Computing
University of Kent
Canterbury, United Kingdom

Thabitha (ta106)
School of Computing
University of Kent
Canterbury, United Kingdom

Rinku Kumari (rk105)
School of Computing
University of Kent
Canterbury, United Kingdom

***Abstract*—HEAT (Haskell Educational Advancement Tool) is a Java Swing IDE for undergraduate Haskell learners. It was not designed with accessibility in mind: a blind user receives no audio feedback, a user with Deuteranopia cannot distinguish the red error text from the green prompt, and a partially sighted user cannot reach the font size control without already being able to read the interface. This paper describes five accessibility extensions added to HEAT during a nine-day group software engineering project: Text-to-Speech (TTS) with plain English error simplification, a Deuteranopia colour blind mode, a font size control panel, a keyboard-navigable startup interface, and three keyboard-driven file dialogs. The extended HEAT enables a blind student to hear every status change and error in plain English, a Deuteranopia student to distinguish console output, and any keyboard-only user to open and create files from the moment the application starts.**

***Keywords*-accessibility, text-to-speech, colour blindness, keyboard navigation, Haskell, IDE, Java Swing**

I. Introduction

HEAT [1] is a Java Swing IDE built for undergraduate Haskell teaching. Its console uses a colour-coded output scheme modelled on WinHugs: the GHCi prompt is rendered in green and compiler errors in red. This was a deliberate usability decision for sighted users, but it is the *only* distinction – there is no audio, no symbol, and no accessible label.

We identified four accessibility barriers:

1) No audio feedback – a blind user cannot tell whether compilation succeeded or failed.
2) Colour-coded output only – red and green are indistinguishable under Deuteranopia.
3) No accessible font size control – partially sighted users cannot enlarge text without navigating menus.
4) Mouse-dependent file navigation – keyboard-only users have no entry point on first launch.

The client, Rogério de Lemos (module convenor), confirmed accessibility for visually impaired users as the project focus and recommended a colour blindness feature. A project consultant also gave mid-week feedback: TTS should speak all GHCi error lines including detail lines, and HEAT should launch maximised. Both were implemented.

Table I
Feature Ownership

Feature	Owner
TTS and error simplification	Yarushah
Keyboard navigation and shortcuts	Yarushah
Keyboard Guide (F1)	Yarushah
Colour blind mode (Deuteranopia)	Thabitha
Font size control	Thabitha
Icon enlargement	Thabitha
Splash screen	Goutham
Keyboard file dialogs (3)	Goutham
Settings integration	Rinku (not delivered)

II. Group Organisation

Table I shows feature ownership. Rinku Kumari was assigned settings integration, text enlargement, and icon enlargement. Her code was not in a working state by the deadline and could not be integrated. Thabitha took over text enlargement and icon enlargement. Settings integration was absorbed into the final merge by Yarushah.

III. Development Process

A. Methodology

We used a lightweight Kanban approach. Features were tracked as cards in Not Started, In Progress, and Done states via group chat with daily informal standups. The project ran for nine days – too short for sprint planning to add value. We chose Kanban over Scrum or RUP because the team was small, the codebase was an existing tool, and requirements were stable after Day 1.

B. Customer Involvement

The client gave verbal guidance on 27 May 2026, recommending a colour blindness feature for at least one category. This shaped our choice of Deuteranopia using the Wong (2011) palette [5]. A project consultant provided additional requirements mid-week: TTS should speak all GHCi error lines including detail lines, and HEAT should launch maximised. Both were implemented.

C. Deviations from Plan

A high contrast mode was dropped at Meeting 2; the Deuteranopia mode covered the colour accessibility requirement. A TTS toggle in the Options menu was deferred and is controlled via `heat.settings`. Rinku's tasks were redistributed after her code could not be integrated.

IV. Requirements

A. Problem Analysis

Blind users rely entirely on audio – HEAT provides none. *Deuteranopia users* cannot distinguish red errors from the green prompt; Coblis [6] simulation confirmed these are visually identical. *Partially sighted users* need larger text but cannot navigate the Options menu to get it if they cannot read the current size.

B. Approaches Considered

For TTS we evaluated FreeTTS [2], Google Cloud TTS, and OS-native speech. OS-native was chosen: no dependencies, offline, works on macOS (`say`), Windows (PowerShell `SpeechSynthesizer`), and Linux (`espeak`).

For colour blind mode, targeted console substitution was preferred over full palette replacement. The console's `JTextPane` applies colour per character run via `StyledDocument`, making targeted substitution sufficient and low-risk.

For font size, an always-visible panel was chosen over settings-only. A user who cannot read the current font size cannot reliably navigate menus to change it.

For file navigation, search-based dialogs replaced `JFileChooser` browsing. A blind user listening to every filename is slow [4]; typing a name fragment and hearing only matching results is much faster.

C. User Stories and Acceptance Criteria

Thirty-two user stories cover five feature areas (US-01–US-32). Table II shows a representative sample. Fifty-six acceptance criteria were defined; 50 pass. AC-28 (responsiveness after Apply) and AC-36 (font size keyboard shortcut) are known limitations. AC-52–AC-56 (Rinku's features) are not tested. AC-12 was tested by setting `TTS_ENABLED=false`: HEAT was confirmed silent.

Table II
Sample Acceptance Criteria

ID	Given	Then	Status
AC-02	TTS on, file open	Speaks success	Pass
AC-08	TTS on, type error	Plain English spoken	Pass
AC-12	TTS disabled	HEAT is silent	Pass
AC-23	Deuteranopia on	Errors render orange	Pass
AC-35	At max 36pt	A+ disabled	Pass
AC-36	Cmd+= pressed	Font size changes	Fail
AC-42	Splash + Cmd+N	TTS confirms, dialog opens	Pass
AC-52	Settings saved	Preferences restored	Not tested

V. Design

A. Software Design

Fig. 1 shows the class structure of the accessibility extension. New classes follow the existing HEAT package conventions. Accessibility utilities are in a new `accessibility` package. Dependencies flow in one direction: `ErrorSimplifier` and `ColorThemeManager` have no external dependencies; `TTSManager` and `KeyboardGuide` depend on `SettingsManager`. `FontSizeManager` depends on `WindowManager` to apply size changes immediately – an accepted architectural trade-off.

B. HCI Rationale

The hardest design problem was file navigation. `JFileChooser` requires visual scanning – unusable for blind users. Our dialogs replace browsing with searching: type a name fragment, hear the result count, navigate by arrow keys. This follows Nielsen's recognition-versus-recall heuristic [3].

The startup interface solves a separate problem: on first launch there is no keyboard entry point. We intercept key events globally via `KeyboardFocusManager.addKeyEventDispatcher()` before the main window is ready, since standard `InputMap`/`ActionMap` bindings require a focused component.

C. Colour Design

Deuteranopia colours were taken from the Wong (2011) peer-reviewed accessible palette [5]: amber `#E69F00` for errors and blue `#0072B2` for the prompt. Both were verified with Coblis [6]. Before-and-after screenshots are in the GitLab wiki.

VI. Implementation

A. TTS (Yarushah)

Fig. 2 shows the full TTS path from GHCi output to spoken speech. `TTSManager` is a singleton using a `BlockingQueue` drained by a background thread. This producer-consumer pattern prevents messages from interrupting each other during rapid status changes. A `startupComplete` flag silences `speak()` during GHCi's startup transitions and is set true after a 2-second delay following `wm.setVisible()`.

`ErrorSimplifier` uses `String.contains()` against 10 GHCi error patterns ordered from specific to general. Unknown errors return `null`, triggering raw-line

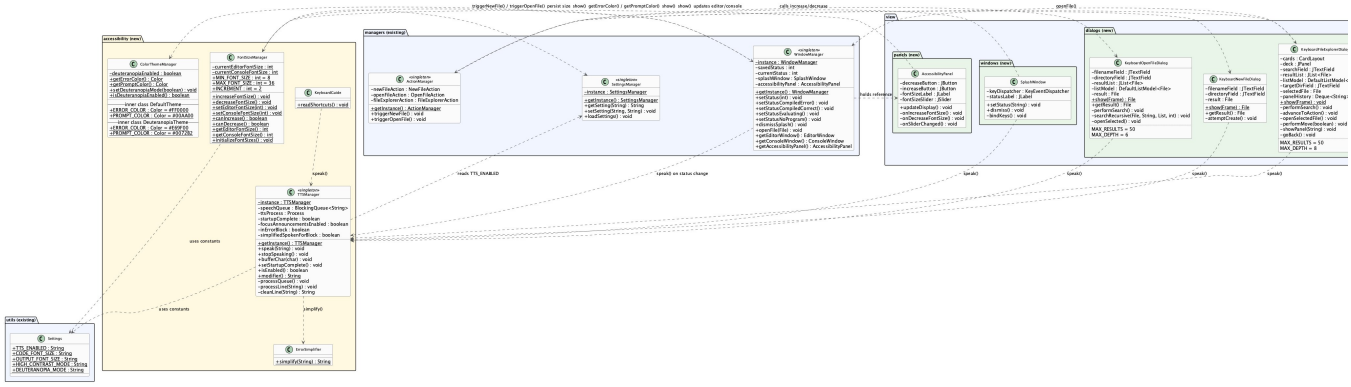

Figure 1. Class diagram of the HEAT accessibility extension showing new `accessibility` package and connections to existing managers, views, and dialogs.

fallback. An `inErrorBlock` state machine speaks every line inside a GHCi error block until a blank line resets it.

B. Colour Blind Mode (Thabitha)

Fig. 3 shows the Cmd+Shift+C toggle sequence. `ColorThemeManager` defines `DefaultTheme` and `DeuteranopiaTheme` as static inner classes. A static boolean selects which theme's colours are returned. `refreshStyles()` re-walks the console document and re-applies colours immediately with no restart needed.

C. Font Size Control (Thabitha)

Fig. 4 shows the font size increase sequence including boundary check, persistence to `heat.settings`, and `AccessibilityPanel` label update. `FontSizeManager` uses static methods clamped to 8–36pt. `AccessibilityPanel` registers listeners on A+, A-, and a `JSlider`; all three paths call the same `FontSizeManager` methods.

D. Splash Screen (Goutham, TTS by Yarushah)

Fig. 5 shows the startup interface activity diagram across four lanes: Main thread, KeyboardFocusManager, TTSManager, and ActionManager. `SplashWindow` is an undecorated `JFrame` with a global `KeyEventDispatcher` for Cmd+N, Cmd+O, and Escape, and a 10-second auto-dismiss timer. Standard `InputMap`/`ActionMap` bindings do not work here because no window has focus.

E. Keyboard File Navigation (Goutham, TTS by Yarushah)

Three keyboard-only dialogs handle file operations. Fig. 6 shows `KeyboardOpenFileDialog`: the user types a fragment, hears the result count, navigates by arrow keys, and presses Cmd+F5 to hear the full path. Fig. 7 shows `KeyboardNewFileDialog`: filename and directory fields with tilde expansion and automatic directory creation. Fig. 8 shows `KeyboardFileExplorerDialog`: a four-panel `CardLayout` workflow for move/copy operations.

F. Settings Integration (Yarushah, absorbed from Rinku)

`Settings.java` adds `TTS_ENABLED`, `HIGH_CONTRAST_MODE`, and `DEUTERANOPIA_MODE` as named constants, ensuring typos are caught at compile time. `SettingsManager` adds defaults (TTS on, both colour modes off) to `createDefaultProperties()`. This work was originally assigned to Rinku Kumari but absorbed into the final merge by Yarushah after Rinku's code could not be integrated.

VII. Quality Assurance

A. Unit Tests

Fifty-one automated unit tests target four pure logic classes:

- `ErrorSimplifierTest` – 13 tests for all error pattern mappings
- `ColorThemeManagerTest` – 9 tests for colour toggle and RGB values
- `FontSizeManagerTest` – 11 tests for boundary enforcement
- `KeyboardDialogsTest` – 18 tests for path normalisation and filename validation

All 51 pass. GUI-dependent classes are covered by functional tests.

B. Functional Testing

Sixteen manual black-box scenarios cover all features. Fourteen pass. FT-10 (Cmd+= font size) and FT-11 (Cmd+- font size) fail: the shortcuts are registered in `ActionManager` but not bound in `DefaultInputHandler`.

C. Code Inspection

Pre-merge inspection confirmed: `BlockingQueue` thread safety in `TTSManager`; no hardcoded colours in `ConsoleWindow`; `ErrorSimplifier` pattern ordering correct; all settings keys use named constants; all `getAccessibleContext()` calls are null-guarded.

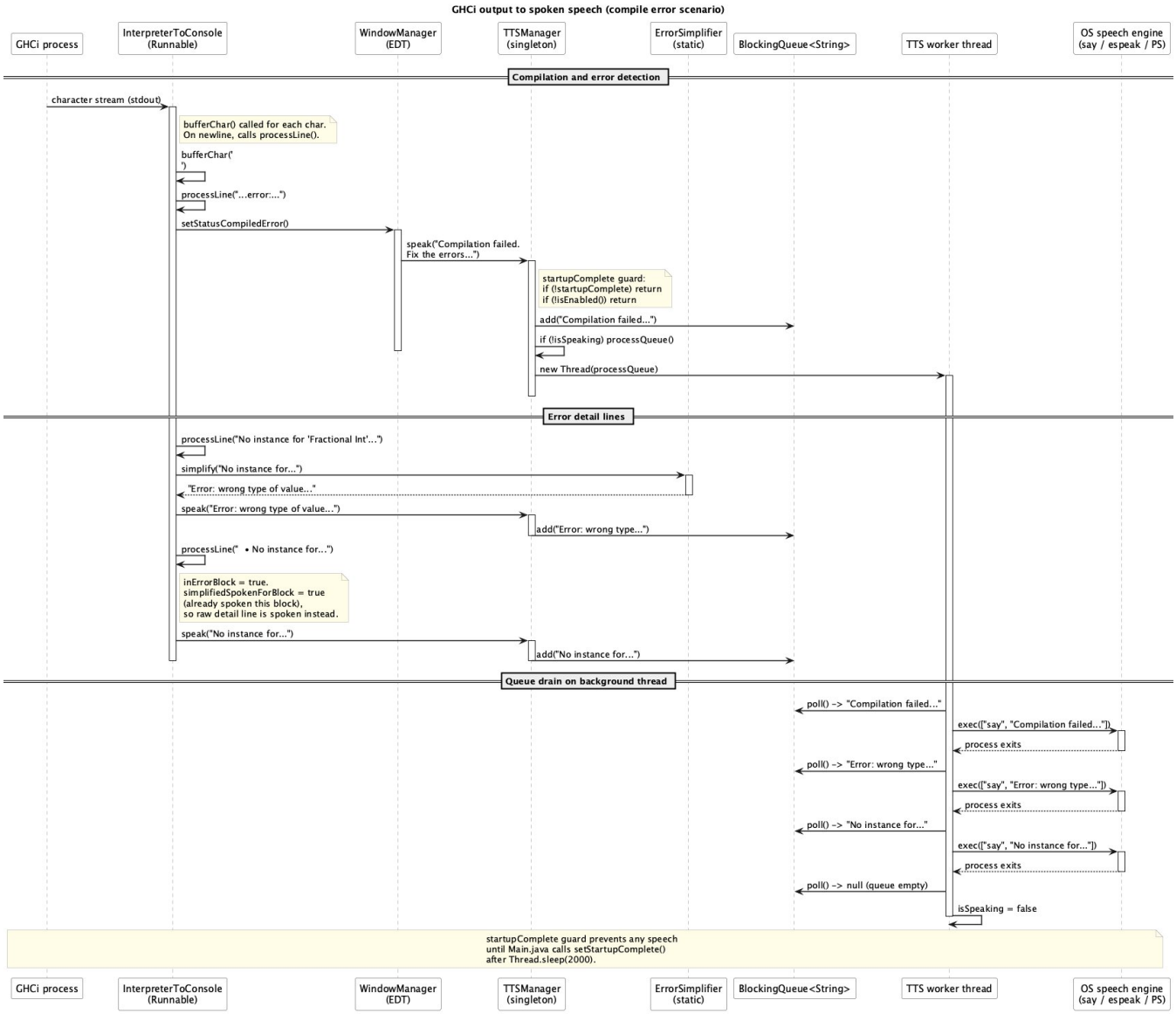

Figure 2. Sequence diagram: TTS path from GHCi output to spoken speech, showing compilation error detection, error detail lines, and queue drain on background thread.

D. Colour Verification

Coblis [6] simulation confirmed that the original red/green colours are visually identical under Deuteranopia, and the amber/blue replacements are clearly distinguishable. Screenshots are in the GitLab wiki.

VIII. Tools

All development used GitLab at https://git.cs.kent.ac.uk/comp7024/g5. Development was done in Visual Studio Code with the Java extension pack. GHCi 9.6.7 was installed via GHCup. macOS VoiceOver was used to verify accessible names on toolbar buttons. Claude Code (Anthropic, Claude Sonnet 4.6) was used for codebase navigation, TTS debugging, merge conflict resolution, and generating the initial `KeyboardFileExplorerDialog` implementation – disclosed per the module AI use policy.

IX. Understanding and Insight

A. What Went Well

Finishing TTS first was the right decision: other members used spoken output to verify their own features. OS-native speech worked immediately with no configuration. The `BlockingQueue` design prevented message overlap during compilation – a problem the first cancel-on-new-message implementation could not solve.

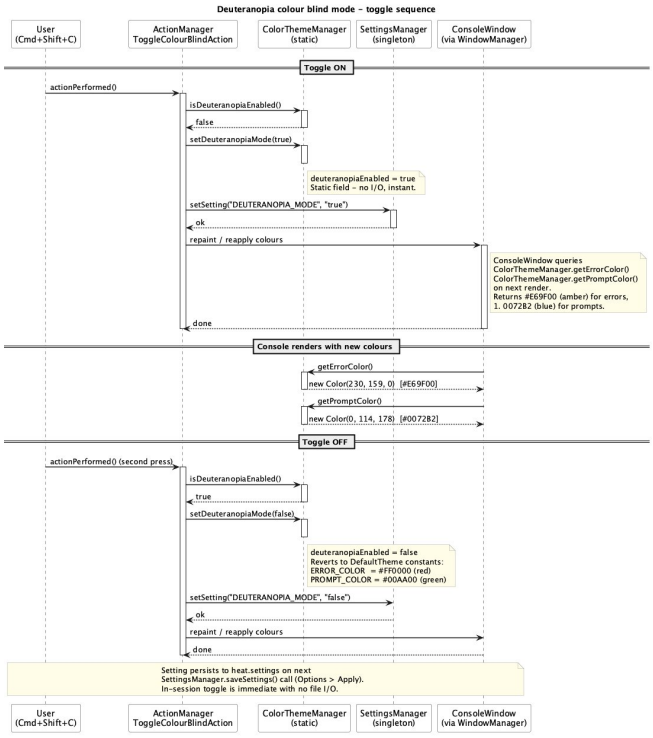

Figure 3. Sequence diagram: Deuteranopia colour blind mode toggle showing ActionManager, ColorThemeManager, SettingsManager, and ConsoleWindow interactions.

B. What Did Not Go Well

The first two days were slower than expected due to unfamiliarity with multi-user GitLab workflows. A merge conflict arose because Goutham pushed a complete codebase snapshot while Yarushah had incremental commits on a different base. The conflict was resolved manually with Claude Code assistance.

Rinku pushed non-functional code directly to the master branch without consulting the group and without the mandatory code review agreed at the kickoff meeting. This introduced broken code into the shared codebase, blocked other members from building, and disrupted the team's ability to push and pull reliably. As a result the team stopped relying on GitLab for sharing work in progress and instead shared files locally, working on them independently and only pushing once features were complete. This is why the final merge was a manual integration of three separate codebases rather than a series of incremental branch merges. Rinku's assigned features were not delivered, requiring Thabitha to take on additional work in the final two days.

C. Lessons Learned

The most concrete lesson was the difference between using Git alone and using Git as a team. Small commits on named branches, frequent merges, and file ownership agreed before work starts are habits that only make sense once you have experienced what happens without them.

We would start integration testing earlier and establish a mid-week check-in requiring each person to demonstrate working code. Testing with an actual visually impaired user was the most significant gap in our process.

X. Conclusions

We extended HEAT with five accessibility features. A blind student can now hear every status change and GHCi error in plain English. A Deuteranopia student can distinguish error text from prompt text. A partially sighted student can increase font size without opening a menu. Any keyboard-only user can create or open a file from the moment the application starts. The most technically interesting feature is the startup interface: a keyboard and TTS entry point before the main Swing window exists, requiring global key event interception rather than standard binding approaches.

Acknowledgment

We thank Rogério de Lemos and the project consultants for guidance during project week.

References

[1] University of Kent, Department of Computing, *HEAT – Haskell Educational Advancement Tool*. Available: https://git.cs.kent.ac.uk/comp7024

[2] Sun Microsystems, *FreeTTS: A Speech Synthesis System Written Entirely in the Java Programming Language*, 2003. Available: https://freetts.sourceforge.net/

[3] J. Nielsen, *Usability Engineering*. Morgan Kaufmann, 1993.

[4] T. V. Raman, *Auditory User Interfaces: Toward the Speaking Computer*. Kluwer Academic Publishers, 1997.

[5] B. Wong, "Points of view: Color blindness," *Nature Methods*, vol. 8, no. 6, p. 441, 2011.

[6] Coblis Colour Blindness Simulator. Available: https://www.color-blindness.com/coblis-color-blindness-simulator/ [Accessed 3 Jun. 2026]

[7] A. Stefik and S. Siebert, "An empirical investigation into programming language syntax," *ACM Trans. Comput. Educ.*, vol. 13, no. 4, pp. 1–40, 2013.

[8] T. V. Raman, "ASTER: Towards modality-independent electronic documents," Ph.D. dissertation, Cornell University, 1994.

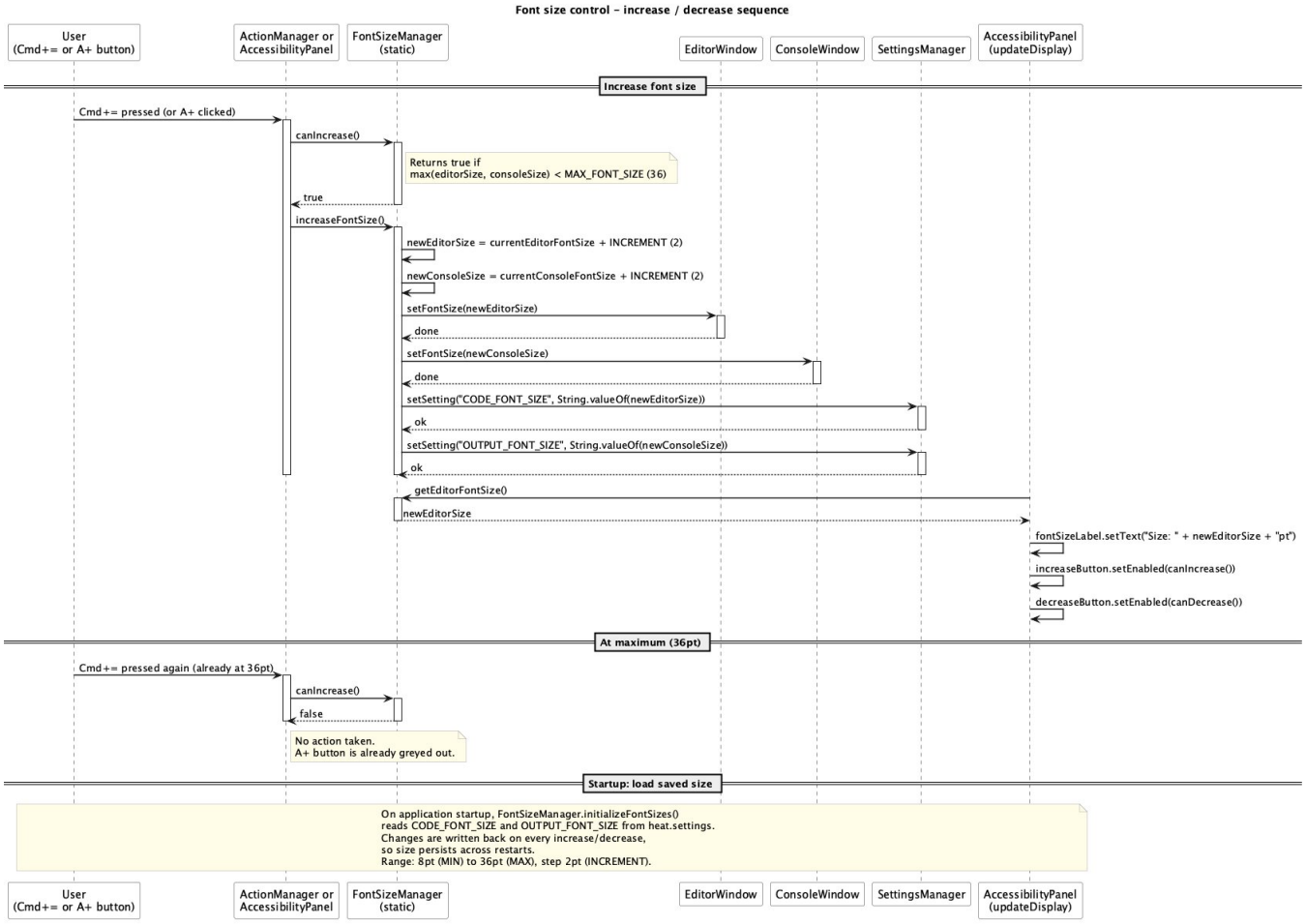

Figure 4. Sequence diagram: font size increase flow showing boundary check, editor/console update, settings persistence, and AccessibilityPanel label refresh.

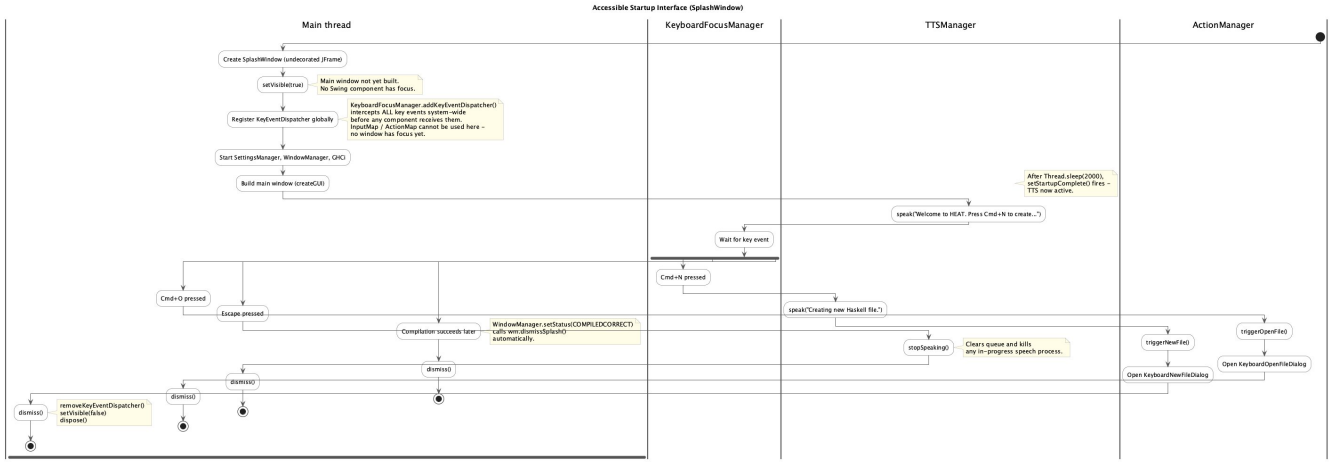

Figure 5. Activity diagram: accessible startup interface (SplashWindow) showing Main thread, KeyboardFocusManager, TTSManager, and ActionManager interactions.

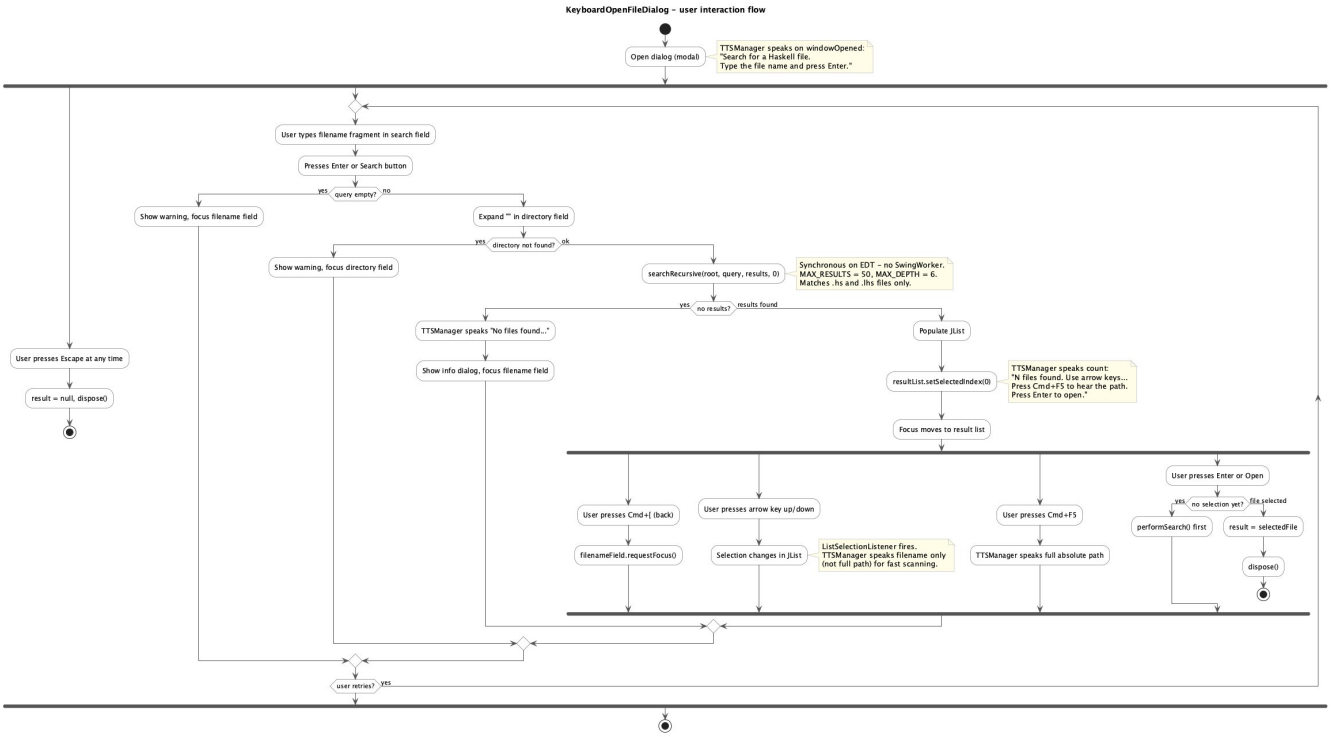

Figure 6. Activity diagram: KeyboardOpenFileDialog interaction flow showing search, TTS result announcement, Cmd+F5 path reading, and file open.

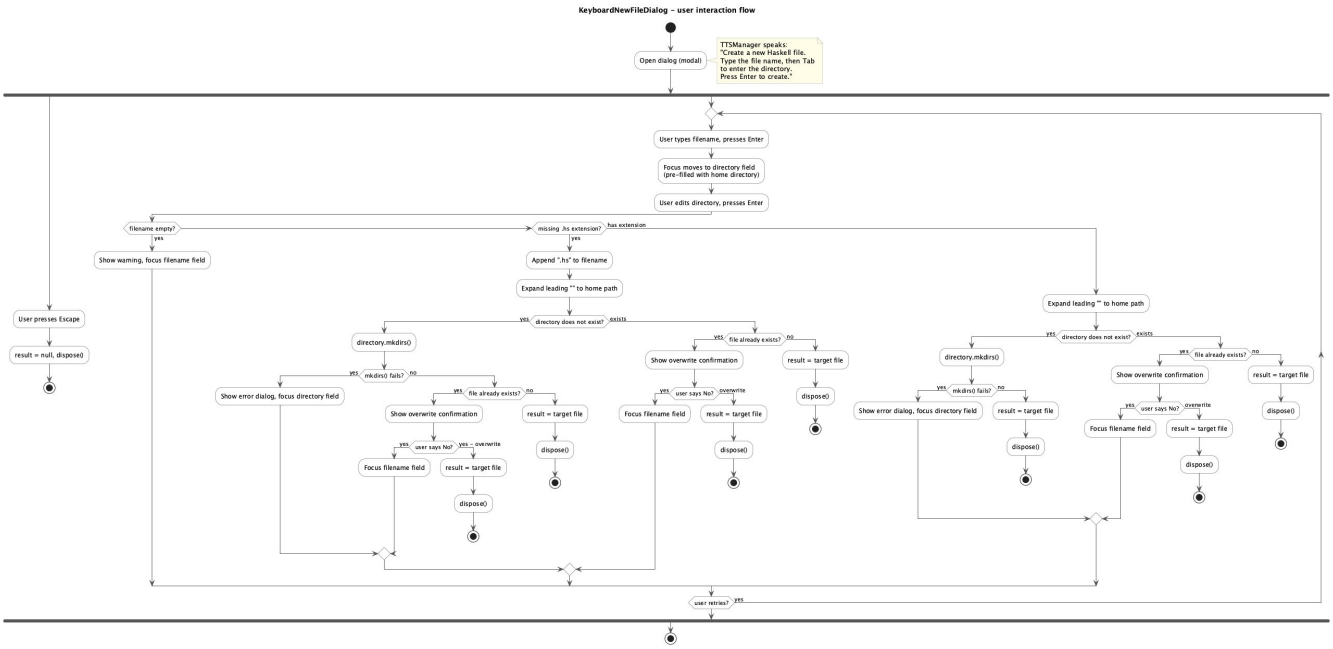

Figure 7. Activity diagram: KeyboardNewFileDialog interaction flow showing filename validation, tilde expansion, directory creation, and overwrite confirmation.

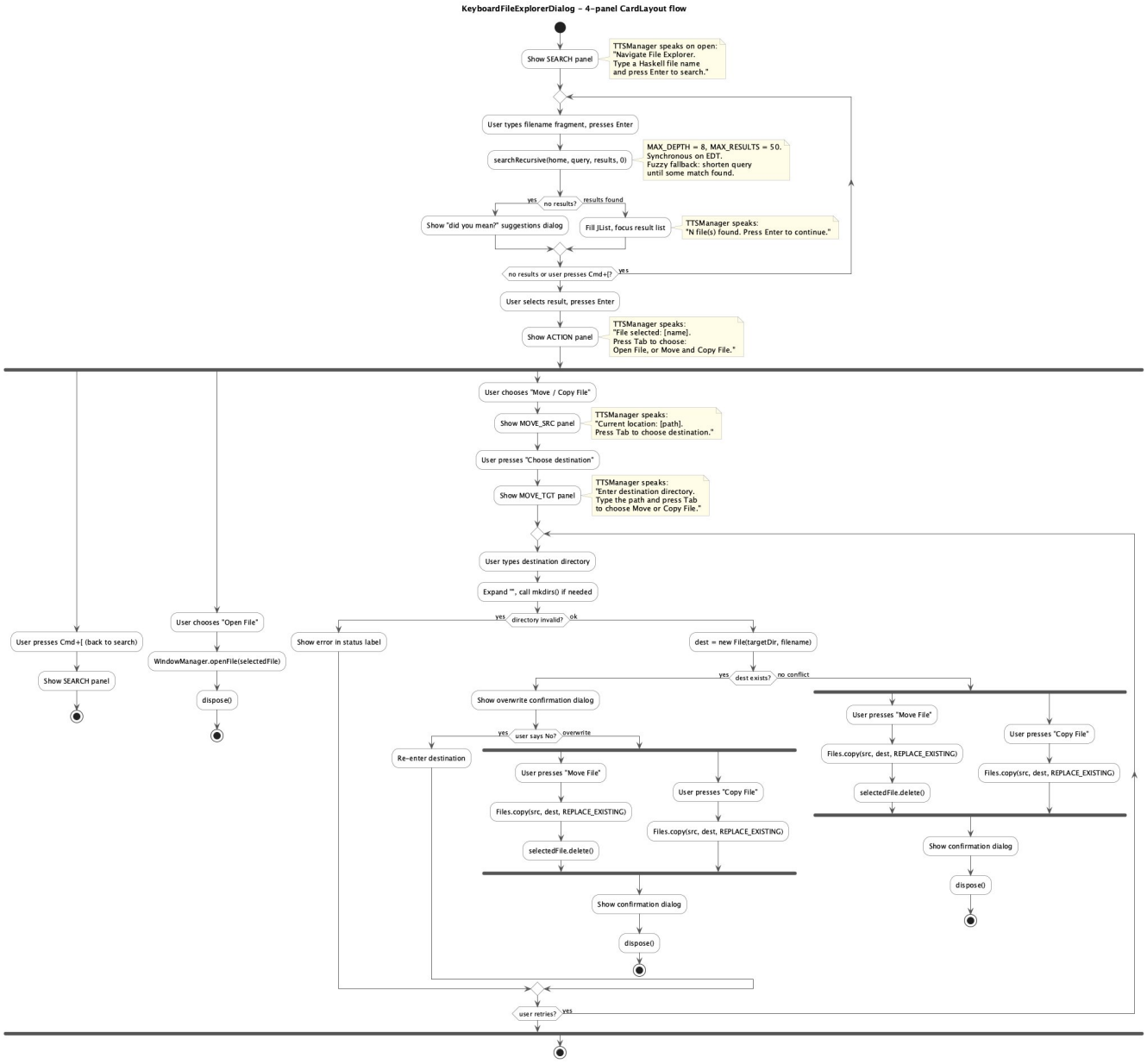

Figure 8. Activity diagram: KeyboardFileExplorerDialog 4-panel CardLayout workflow for search, action selection, move source confirmation, and destination entry.